

Static vs Non-Static Constructors in C# with Examples

In this article, I am going to discuss **Static vs Non-Static Constructors in C#** with Examples. Please read our previous article, where we discussed [Why We Need Constructors in C#](#) with Examples.

Static vs Non-Static Constructors in C#

Point1:

If a constructor is explicitly declared by using the static modifier, we call that constructor a static constructor whereas the rest of the others are called non-static constructors only. For a better understanding, please have a look at the below code. Non-Static Constructors are also called Instance Constructors in C#.

```
public class ConstructorsDemo
{
    //Static Constructor
    static ConstructorsDemo()
    {
        Console.WriteLine("Static Constructor is Called");
    }

    //Non-Static Constructor
    public ConstructorsDemo()
    {
        Console.WriteLine("Non-Static Constructor is Called");
    }
}
```

Point2:

Constructors are responsible for initializing the fields or variables of a class. Static Fields/Variables are initialized by static constructors and non-static fields/variables are initialized by non-static or instance constructors in C#. For a better understanding, please have a look at the below code. Here, both the variables x and y are initialized with the default value i.e. 0. The variable x is going to be initialized by a static constructor whereas the variable y is going to be initialized by a non-static constructor.

```

public class ConstructorsDemo
{
    static int x; //It is going to be initialized by static constructor
    int y; //It is going to be initialized by non-static constructor

    //Static Constructor
    static ConstructorsDemo()
    {
        //This constructor initialized the static variable x with default value i.e. 0
        Console.WriteLine("Static Constructor is Called");
    }

    //Non-Static Constructor
    public ConstructorsDemo()
    {
        //This constructor initialized the static variable y with default value i.e. 0
        Console.WriteLine("Non-Static Constructor is Called");
    }
}

```

Point3:

Static constructors are implicitly called whereas non-static constructors are explicitly called. For better understanding, please have a look at the below example. Here, the Program execution is always going to start from the Main method. In the below example, the Main method and Static constructors, both are present in the same classes. So, before executing the body of the Main method, it will execute the static constructor of the class because the static constructor is the first block of code to be executed under a class and once the static constructor execution is completed, then it will execute the Main method body. So, when you will run the below code, you will see that first, the static constructor is executed and then only the Main method is executed.

```

using System;
namespace ConstructorDemo
{
    public class ConstructorsDemo
    {
        static int x; //It is going to be initialized by static constructor
        int y; //It is going to be initialized by non-static constructor
        //Static Constructor
        static ConstructorsDemo()
        {
            //This constructor initialized the static variable x with default
            Console.WriteLine("Static Constructor is Called");
        }
        //Non-Static Constructor
        public ConstructorsDemo()
        {
            //This constructor initialized the static variable y with default

```

```

        Console.WriteLine("Non-Static Constructor is Called");
    }
    //Main Method is the Entry Point for our Application Execution
    static void Main(string[] args)
    {
        //Before Executing the body of Main Method, Static Constructor is
        Console.WriteLine("Main Method Body..");
        Console.ReadKey();
    }
}

```

Output:

```

Static Constructor is Called
Main Method Body..

```

If you notice, we did not call the Static Constructor anywhere in our code, but it is executed. That means, it is always going to be called Implicitly. In the above example, we have not called the no-static constructors, and hence the non-static constructor is not executed.

Point4:

Static Constructors execute immediately once the execution of a class start and moreover, it is the first block of code to run under a class whereas non-static constructors execute only after creating the instance of the class as well as each and every time the instance of the class is created.

For a better understanding, please have a look at the below example. In the below example, the Main method and Static constructors are present in two different classes. So, the Program execution started from the Main method and it will start executing the Main method body. Then inside the Main method, we are creating the instance of ConstructorsDemo class i.e. we are trying to execute ConstructorsDemo class for the first time and as this class has a static constructor, that static constructor will implicitly be called and once that static constructor completes its execution, then only the instance is being created and the non-static constructor is executed.

```

using System;
namespace ConstructorDemo
{
    class Program
    {
        //Main Method is the Entry Point for our Application Execution
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            //Creating Object of ConstructorsDemo
            //Now the ConstructorsDemo class Execution Start
            //First, it will execute the Static constructor
            //Then it will execute the non-static constructor
            ConstructorsDemo obj = new ConstructorsDemo();
            Console.WriteLine("Main Method Completed");
        }
    }
}

```

```

        Console.ReadKey();
    }
}
public class ConstructorsDemo
{
    static int x; //It is going to be initialized by static constructor
    int y; //It is going to be initialized by non-static constructor
    //Static Constructor
    static ConstructorsDemo()
    {
        //This constructor initialized the static variable x with default
        Console.WriteLine("Static Constructor is Called");
    }
    //Non-Static Constructor
    public ConstructorsDemo()
    {
        //This constructor initialized the static variable y with default
        Console.WriteLine("Non-Static Constructor is Called");
    }
}
}

```

Output:

```

Main Method Started
Static Constructor is Called
Non-Static Constructor is Called
Main Method Completed

```

In the above example, the execution takes place as follows:

1. First, the Main method of the Program class starts its execution as it is the entry point for our application.
2. Then the Static Constructor of the ConstructorsDemo class is executed.
3. Then the Non-Static Constructor of the ConstructorsDemo class is executed.
4. Finally, the Main method completes its execution.

Point5:

Static Constructors are executed only once whereas the non-static constructors are executed 0 or n number of times depending on the number of instances we created for the class. For a better understanding, please have a look at the below example. In the below example, the place where we try to invoke the static variable using the class name ConstructorsDemo, first it will invoke the static constructor implicitly. As we are not creating an instance for the ConstructorsDemo class, the non-static constructor is not going to be executed.

```

using System;
namespace ConstructorDemo

```

```

{
    class Program
    {
        //Main Method is the Entry Point for our Application Execution
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            //As soon as it finds ConstructorsDemo.x,
            //it will first execute the static constructor of the class
            Console.WriteLine(ConstructorsDemo.x);

            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
    }
    public class ConstructorsDemo
    {
        public static int x; //It is going to be initialized by static constructor
        public int y; //It is going to be initialized by non-static constructor
        //Static Constructor
        static ConstructorsDemo()
        {
            //This constructor initialized the static variable x with default value 0
            Console.WriteLine("Static Constructor is Called");
        }
        //Non-Static Constructor
        public ConstructorsDemo()
        {
            //This constructor initialized the static variable y with default value 0
            Console.WriteLine("Non-Static Constructor is Called");
        }
    }
}

```

Output:

```

Main Method Started
Static Constructor is Called
Value of x: 0
Main Method Completed

```

Now, please have a look at the below example. Here, we are creating 3 instances of the ConstructorsDemo class. In this case, when we create the first instance, before executing the non-static constructor, it will execute the static constructor first. Once the static constructor is executed, then the non-static constructor starts executing. This happens only for the first instance of creation time. From the second instance creation, the static constructor is not going to be executed. So, when you will run the above code, you will see that the static constructor is executed only once and the non-static constructor is executed three times.

```

using System;
namespace ConstructorDemo
{
    class Program
    {
        //Main Method is the Entry Point for our Application Execution
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started");
            //Before Executing the non-static constructor
            //it will first execute the static constructor of the class
            ConstructorsDemo obj1 = new ConstructorsDemo();
            //Now, onwards it will not execute the static constructor,
            //Because static constructor executed only once
            ConstructorsDemo obj2 = new ConstructorsDemo();
            ConstructorsDemo obj3 = new ConstructorsDemo();
            Console.WriteLine("Main Method Completed");
            Console.ReadKey();
        }
    }
    public class ConstructorsDemo
    {
        public static int x; //It is going to be initialized by static constructor
        public int y; //It is going to be initialized by non-static constructor
        //Static Constructor
        static ConstructorsDemo()
        {
            //This constructor initialized the static variable x with default value 0
            Console.WriteLine("Static Constructor is Called");
        }
        //Non-Static Constructor
        public ConstructorsDemo()
        {
            //This constructor initialized the static variable y with default value 0
            Console.WriteLine("Non-Static Constructor is Called");
        }
    }
}

```

Output:

```

Main Method Started
Static Constructor is Called
Non-Static Constructor is Called
Non-Static Constructor is Called
Non-Static Constructor is Called
Main Method Completed

```

When the Static Constructor of a Class is Executed in C#?

It is very important for us to understand when the static constructor of a class is executed implicitly. The following are the three scenarios where the static constructor is executed implicitly.

1. If both the static constructor and Main method are present in a single class, then the program execution will start from the Main method, but before executing the Main method body, it will first execute the Static Constructor of the class.
2. When we call any static variables or static methods for the First time in a class, then it will execute the static constructor of that class.
3. When we create an instance of a class for the first time, then before executing the non-static constructor, it will first execute the static constructor of that class.

The most important point that you need to remember is static constructors are executed only once irrespective of the number of times you called the static variables or static methods or irrespective of the number of times you created the instance of the class.

Note: In the life cycle of a class (life cycle in the sense the moment we start the execution to the end of the class is considered as one life cycle), the static constructor is executed once and only one time whereas the non-static constructors execute for 0 times if no instances are created and n times if n number of instances are created.

Point6:

Non-Static Constructors can be parameterized whereas the static constructors cannot have any parameter. This is because we explicitly call the non-static constructors, so we can have a chance to pass parameters. On the other hand, static constructors are implicitly called and it is the first block of code to run under a class, and hence we don't have any chance to pass parameters. For a better understanding, please have a look at the below code. So, a static constructor must be parameterless in C#.

```

public class ConstructorsDemo
{
    public static int x; //It is going to be initialized by static constructor
    public int y; //It is going to be initialized by non-static constructor

    //Non-Static Constructor
    public ConstructorsDemo(int y)
    {
        //This constructor initialized the static variable y with default value i.e. 0
        Console.WriteLine("Non-Static Constructor is Called");
    }

    //Static Constructor
    static ConstructorsDemo(int x)
    {
        //This constructor initialized the static variable x with default value i.e. 0
        Console.WriteLine("Static Constructor is Called");
    }
}

```

ConstructorsDemo.ConstructorsDemo(int x) e. 0

'ConstructorsDemo.ConstructorsDemo(int)': a static constructor must be parameterless

Point7:

Non-Static Constructors can be overloaded whereas static constructors cannot be overloaded. Overloading is something that comes into the picture based on the parameters. As we have a chance to pass parameters in the case of non-static constructors, overloading is possible. On the other hand, we cannot pass parameters to static constructors i.e. static constructors are parameterless, and hence overloading is not possible. For a better understanding, please have a look at the below code.

```

public class ConstructorsDemo
{
    //Non-Static Constructor 1, with 0 Parameter
    public ConstructorsDemo()
    {
    }

    //Non-Static Constructor 2, with 1 Parameter
    public ConstructorsDemo(int y)
    {
    }

    //Non-Static Constructor 3, with 2 Parameter
    public ConstructorsDemo(int x, int y)
    {
    }

    //Static Constructor is Parameterless
    static ConstructorsDemo()
    {
    }
}

```

↓ Overloading Not Allowed

```

//Trying to Overload Static Constructor, getting Compiletime error
static ConstructorsDemo(int x)
{
}

```

ConstructorsDemo.ConstructorsDemo(int x)

'ConstructorsDemo.ConstructorsDemo(int)': a static constructor must be parameterless

Point8:

Every class contains an implicit constructor if not defined explicitly and those implicit constructors are defined based on the following criteria.

1. Every class except a static class contains an implicit non-static constructor if not defined with an explicit constructor.
2. Static constructors are implicitly defined only if that class contains any static fields or else that constructor will not be present provided that the class does not have an explicit static constructor.

Summary of Static and Non-Static Constructors:

1. A constructor is a special method inside a class used to initialize the data members. If we create the constructor using a static modifier then we call it a static constructor and the rest of all are non-static constructors only.
2. The static constructor is used to initialize the static data members and the non-static constructor is used to initialize the non-static data members of a class.
3. The static constructor is always invoked implicitly while the non-static constructor is always invoked explicitly.

4. If we have not defined any constructor explicitly, then the compiler will provide the implicit constructor in the following conditions.
5. For a static class, the compiler will provide a static constructor implicitly, but no non-static constructor.
6. For a non-static class, the compiler will provide a non-static constructor, if the non-static class has any static member, then only the compiler will provide the static constructor.
7. Static constructors will execute only once during the life cycle of a class and non-static constructors are executed 0 or n number times. If we have not created any object, then the constructor will execute 0 times and if we create n number of objects, then the constructor will execute n number of times.
8. In a class, we can have only one static constructor and i.e. too parameterless, and hence static constructor cannot be overloaded. But, in a class, we can define any number of non-static constructors and hence non-static constructors as overloaded.
9. A static constructor is executed when our class execution starts and it will execute only once and it will be the first block inside a class to be executed while non-static constructors are going to be executed when we create an instance of a class and for each instance of the class.

So, these are all the differences between static and non-static constructors in C#. To learn more about constructors, please see the following.

[Constructors in C#](#)

[Type of Constructors in C#](#)

[How to use Constructors in Real-time Application Development using C#](#)

In the next article, I am going to discuss [Private Constructors in C#](#) with Examples. Here, in this article, I try to explain **Static vs Non-Static Constructors in C#** with Examples. I hope you enjoy this Static vs Non-Static Constructors in C# with Examples article. Please give your feedback, suggestions, and questions about this Static vs Non-Static Constructors in C# article in the comment section.